ML OPS

Antony Schutz 2026

UV

Installation

windows:

```
powershell -c "ir https://astral.sh/uv/install.ps1 | iex"
```

→ ir ou irm

mac/linux :

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
brew install uv
```

Commande UV

```
uv init my-ml-project    # initialise le projet
uv venv my_venv          # Création du venv my_venv
uv python install 3.12   # Installe la version 3.12
uv python list           # Liste toutes les versions installées
uv python pin 3.12       # Force le projet actuel à utiliser la 3.12
uv add pandas            # ajoute pandas au projet (.toml et uv.lock)
uv add --dev pytest      # ajoute pytest pour le dev pas pour la prod
uv run python train.py   # execute un script
                         # uvx est pour uv eXecute
uvx ruff check .         # Exécute ruff même s'il n'est pas dans le projet
uvx mlflow ui            # Lance l'interface MLflow instantanément
```

UV INIT

Va créer:

- le fichier projet **`pyproject.toml`** à éditer
- le dossier **`.git`** qu'il faudra éditer ou supprimer
- le fichier **`.gitignore`** avec des valeurs par défaut
- le fichier **`README.md`** vierge
- le fichier **`uv.lock`** qui est l'analogue du requirements.txt

UV TOOLS

Fichier projet pyproject.toml (auto)

```toml
[project]
name = "mon-projet-ia"              # Nom du package (utilisé pour l'install)
version = "0.1.0"                   # Version logicielle (SemVer : Major.Minor.Patch)
description = "Formation MILOps"    # Courte description pour le catalogue
readme = "README.md"                # Fichier qui sert de page d'accueil
requires-python = ">=3.12"          # Version de Python minimale exigée
dependencies = [                    # Ce qui est OBLIGATOIRE pour la PROD
    "pandas>=2.2.0",                # Dataframes (version min pour sécurité)
    "scikit-learn",                 # Modèles ML
    "mlflow",                       # Tracking des expériences
]
[dependency-groups]
```

Fichier projet `pyproject.toml` (auto)

```
[dependency-groups]
dev = [                          # Ce qui sert uniquement au DÉVELOPPEUR
    "pytest>=8.0.0",             # Framework de tests
    "pytest-cov",                # Calcul de la couverture de test (%)
    "ruff",                      # Linter et formateur ultra-rapide
    "sphinx",                    # Générateur de documentation HTML
    "furo",                      # Thème moderne et joli pour Sphinx
]
```

Fichier projet `pyproject.toml` (manuel)

```toml
[tool.ruff]                          # Configuration de l'outil RUFF
line-length = 88                     # Taille max d'une ligne (standard Black)
target-version = "py312"             # On vise une syntaxe Python 3.12

[tool.ruff.lint]                     # Règles de nettoyage du code
select = [
    "E",    # pycodestyle (erreurs de style standard)
    "F",    # Pyflakes (erreurs de logique comme variables oubliées)
    "I",    # isort (range les imports par ordre alphabétique)
    "N",    # pep8-naming (vérifie que les classes sont en Majuscule, etc.)
]
```

Mais ça ne s'arrête pas là

On peut :

- avoir des groupes de dépendances.
  ⟶ `uv sync --group ia`
  ⟶ `uv pip install --group ia`
- configurer les tests
  ⟶ `term : pytest`
- avoir des scripts personnalisés
  ⟶ `term : api-data`

`uv add --group api fastapi uvicorn`

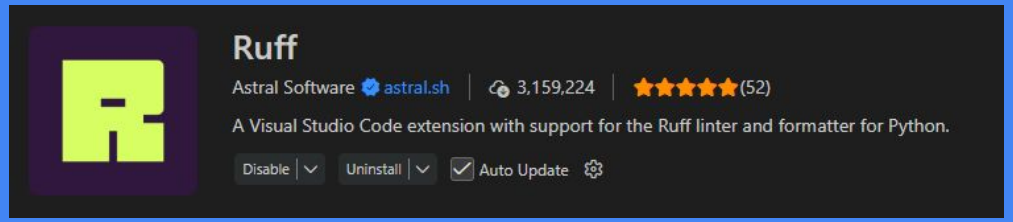
```
[dependency-groups]
dev = [
    "pytest>=8.0.0",
    "pytest-cov",
    "ruff",
]
data = [
    "fastapi",
    "sqlalchemy",
]
ia = [
    "mlflow",
    "scikit-learn",
]

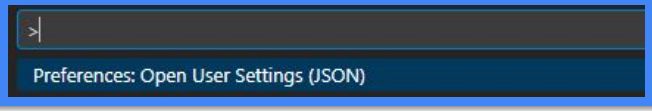
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = "test_*.py"
# --cov permet de générer le rapport de couverture automatiquement
# --cov-report term-missing affiche les lignes non testées dans le terminal
addopts = "-v --cov=app_data --cov=app_ia --cov-report=term-missing"

[tool.coverage.run]
omit = ["tests/*", "**/__init__.py"]

[project.scripts]
# Format : nom_commande = "module.fichier:nom_fonction"
api-data = "app_data.main:start_server"
api-ia = "app_ia.main:start_server"
front = "app_front.main:start_front"
```

RUFF - automatique





ouvrez le fichier `setting.json` dans vs code (shift+ctrl+P cherchez settings)

Après installation (`Shift+Alt+F`) pour choisir `Ruff`

Dans setting user json

Attention une seule clé [`python`] possible

```
    "workbench.colorTheme" : "Default Dark+",
    "[python]": {
        "editor.formatOnType" : true,
        "editor.formatOnSave" : true,
        "editor.defaultFormatter" : "charliermarsh.ruff",
        "editor.codeActionsOnSave" : {
        "source.fixAll.ruff" : "explicit",
        "source.organizeImports.ruff" : "explicit"
    }
    },
    "terminal.integrated.defaultProfile.windows" : "PowerShell",
```

Vous avez un linting automatique quand vous sauvegardez le fichier 😓

Docstring Module Level

On met un docstring qui définit le module en en-tête dans le fichier du module

```
"""Traitement des données de ventes.
Ce module contient les fonctions de ...
Exemple:
    $ uv run python src/data_processing.py
Attributes:
    MIN_PRICE (float): Le prix
"""

import pandas as pd
import numpy as np

MIN_PRICE = 0.01

def ma_fonction()...
```

Docstring Function Level

On met un docstring dans chaque fonctions qui résume la fonction, ses arguments et ses retours

(Google Norm)

```
def calculate_grade_average(grades):
    """Calcule la moyenne pondérée des notes.

    Args:
        grades (list[float]): Une liste des notes obtenues
par l'étudiant.

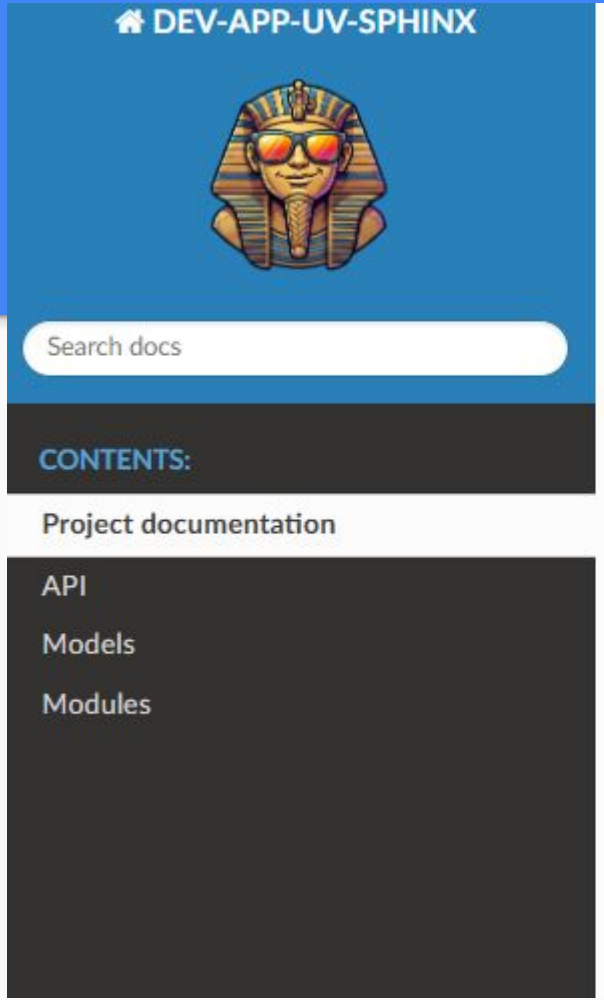
    Returns:
        float: La moyenne pondérée calculée.

    Raises:
        ValueError: Si la liste des notes est vide.
    """
    if not grades:
```

Live DEMO

Intégration sphinx

auto doc





[IA DOCS Sphinx]

Test

Installation:

```
uv add --dev pytest pytest-cov
```

Les fichiers doivent commencer par `test_xxx.py`

Exécuter les tests:

```
uv run pytest
```

Structure:

```
projet/
├── application/
│   └── main.py              # l'application
├── tests/
│   ├── __init__.py          # (Optionnel)
│   └── test_main.py         # Les tests
├── pyproject.toml
└── uv.lock
```

Le dossier de test est à la racine

exemple de test

```python
# application/main.py
def add_bonus(score: float) -> float:
    return score + 2.0

# tests/test_processing.py
from processing import add_bonus

def test_add_bonus():
    # Arrange & Act
    result = add_bonus(10.0)
    # Assert
    assert result == 12.0
```

`assert result == 12.0`

retournera une erreur si `result` est différent de `12`

Liste non exhaustive des tests pytest

```
assert result == expected
assert 0.1 + 0.2 == pytest.approx(0.3)

# Le test va se lancer 3 fois automatiquement.
@pytest.mark.parametrize("input_val, expected", [
    (10, 12),
    (20, 22),
    (0, 2),
])
def test_add_bonus_multiple(input_val, expected):
    assert add_bonus(input_val) == expected
```

Fixtures

```python
# Si on a besoin d'une base de données ou d'un gros CSV pour les
# tests, on ne va pas le charger dans chaque fonction. Tu crées une Fixture.
@pytest.fixture
def mock_df():
    """Prépare un petit DataFrame de test."""
    return pd.DataFrame({"price": [10, 20], "qty": [1, 2]})

def test_total_revenue(mock_df): # On "injecte" la fixture ici
    assert mock_df["price"].sum() == 30
```

Test Coverage

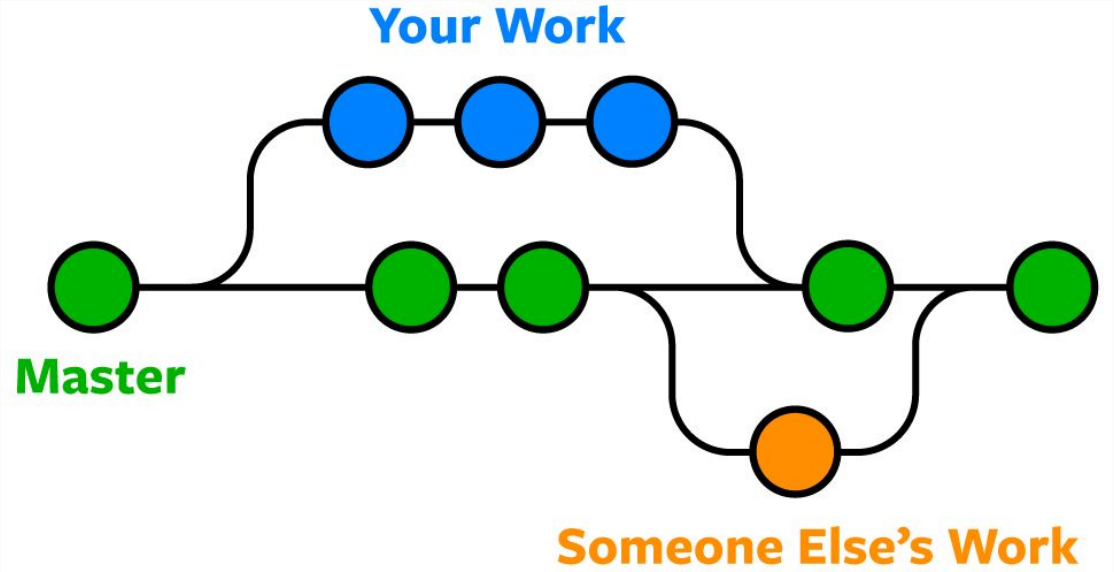
```
uv run pytest --cov=src tests/
uv run pytest --cov=src --cov-report=html tests/
```

Pytest coverage (pytest-cov) permet de savoir la proportion de code testé et génère un rapport qui est compris par GitHub qui l'affiche dans votre dépôt.

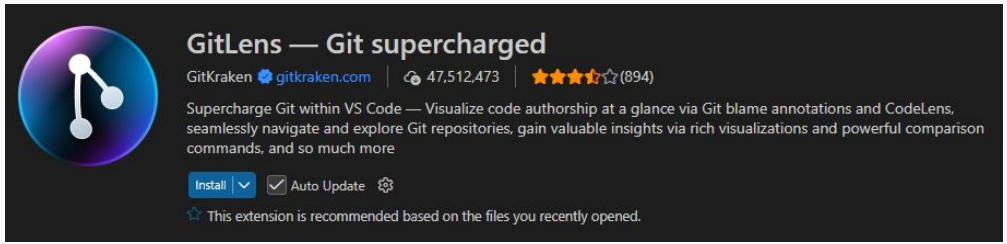
C'est la grande classe! Il peut aussi générer un HTML qui montre en rouge les parties de code non testé

GitHub Branching

Le branching est très important lors du développement d'un application en équipe. On ne mélange pas les branches de développement et celle du master

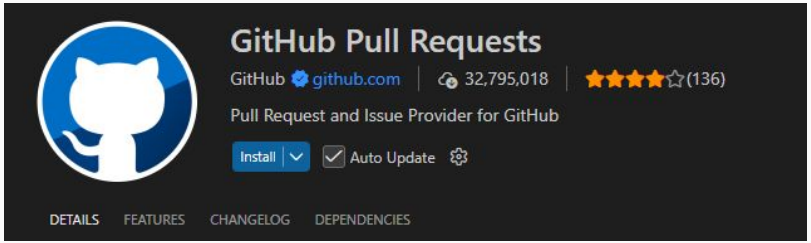


100 % VSCODE



Permet de savoir qui a fait une modification dans le code

Permet de gérer les pull requests directement dans vscode



[IA Branching - Vscode - GitHub]

DEMO

GitHub Action et Intégration Continu (CI)

On peut configurer GitHub action pour qu'il exécute les tests à chaque fois qu'un push est effectué sur une branche ou le master.

Il faut créer un fichier de config (yaml)

```
mon-projet-ia/
├── .github/
│   └── workflows/
│       └── ci.yml
```

Structure du CI.yml

```yaml
name: CI Quality Check

on:
  push:
    branches: [ main, dev ]    # Se déclenche quand on push sur dev ou main
  pull_request:
    branches: [ main ]         # Se déclenche quand on veut merger vers
main
```

Structure du CI.yml

```yaml
jobs:
  test-and-lint:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Install uv
        uses: astral-sh/setup-uv@v3
        with:
          enable-cache: true
```

Vérifier les versions pour 2026

Structure du CI.yml

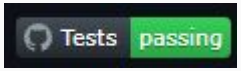
```yaml
      - name: Set up Python
        run: uv python install 3.12

      - name: Lint with Ruff
        # On fait le check (erreurs) et le format (style)
        run: |
          uv run ruff check .
          uv run ruff format --check .

      - name: Run Tests with Pytest
        # On lance les tests sur le dossier tests/ en ciblant src/
        run: uv run pytest --cov=src tests/
```

récupérer le badge github pour les tests

```
[![Build
Status](https://github.com/PSEUDO/DEPOT/actions/workflows/build_tests.yml/badge.svg)](ht
tps://github.com/PSEUDO/DEPOT/actions)
```



DOCKER



Conteneurisation - DOCKER

Installation de docker desktop (docker/docker-compose/kubernetes)

https://www.docker.com/products/docker-desktop/

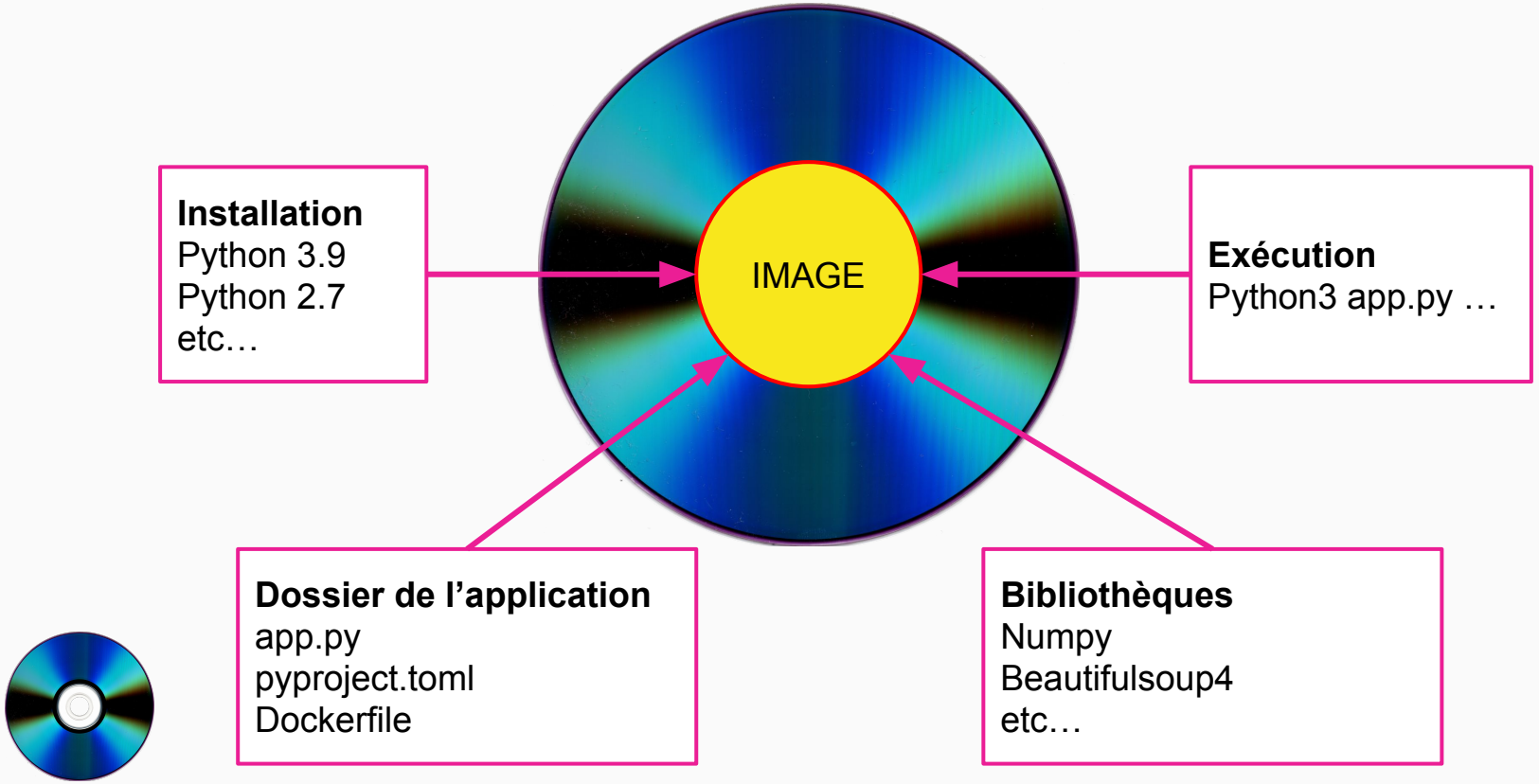
Vérifier la version : `docker -v`





Comme Github héberge les codes, Docker Hub héberge les dockers Docker Hub

Une image Docker est une capture instantanée d'un conteneur



Dockerfile avec UV

Pour exécuter un conteneur `docker` il faut tout d'abord une IMAGE

L'image est construite à partir d'un `Dockerfile` (sans extension)



https://configs.sh/dockerignore/

.gitignore generator

.dockerignore

Tout comme pour git, docker possède aussi son ignore, cela évite de surcharger l'image docker et/ou de devoir rendre compliqué le Dockerfile

Et aussi de ne pas copier des choses inutiles comme le .venv, la doc, le.git etc…

```
# .dockerignore
.git
.github
.venv
tests/
docs/
htmlcov/
.pytest_cache/
.ruff_cache/
```

Dockerfile

Les commandes les plus importantes des Dockerfile

`FROM`	**Créée le noyau que le conteneur fera tourner**
`RUN`	**Exécute une commande**
`WORKDIR`	**Change le répertoire de travail (si le répertoire n'existe pas, il le crée)**
`COPY`	**Copie un ou plusieurs fichier(s) vers une destination**
`EXPOSE`	**Définit le port de l'application**
`CMD`	**Le code à exécuter et comment l'exécuter**

Références docker

Les bonnes pratiques

Dockerfile

```
# La version de python
FROM python:3.12-slim

# On récupère uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/

# on crée un dossier pour l'application
WORKDIR /app

# On copie uniquement les fichiers de dépendances
COPY pyproject.toml uv.lock ./

# On installe les paquets (uv sync -> .venv)
RUN uv pip install --system -r pyproject.toml

# On copie le reste
COPY . .

# On exécute le code
CMD ["python", "main.py"]
```

Docker Hub

A vous de tester!

- Créez un dossier vscode
- Créez un main.py
- `uv init`
- `uv add numpy`
- Créez le Dockerfile
- `docker build <>/<>:V0`
- `docker run <>/<>:V0`
- `docker login`
- `docker push <>/<>:V0`

main.py

```
import numpy as np

print("Whalecome Docker")

print(np.arange(10))
```



Docker Hub

Commandes docker

docker build -t <nom>:<tag> .

Crée l'image si un Dockerfile se trouve dans le dossier avec le nom <nom> et la version <tag>
Une bonne pratique est de mettre <pseudo>/<nom>:<tag> ou pseudo est votre pseudo de docker hub

docker run -d -p 8000:5000 --rm <nom>:<tag>

fait tourner l'image, vers le port 8000, en détaché (-d)

docker images

Affiche les images en cours

docker container ls

Affiche les conteneur en cours

```
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS                    NAMES
dc776853567f   devermyst/ktfda    "python /code/app.py"    2 minutes ago    Up 2 minutes    0.0.0.0:8000->5000/tcp   vigorous_williams
```

Docker Hub

Commandes docker

docker stop/restart <CONTAINER ID>

Stop l'exécution du/restart container ayant l'ID: CONTAINER ID

docker rmi <Image ID>

Supprime l'image ayant l'ID: IMAGE ID

docker rmi -f $(docker images -aq)

Supprime toutes les images

docker tag <nom>:<tag> <nouveau nom>:<nouveau tag>

Change le nom et tag

Docker Hub

Commandes docker

docker login

Ouvre l’accès à docker hub

docker push <pseudo>/<nom>:<tag>

Push l’image sur docker hub -- <pseudo> obligatoire

docker pull <pseudo>/<nom>:<tag>

Pull l’image depuis docker hub

docker system prune -a

Réinitialise le système, détruit toutes les images et conteneur

Docker Hub

Commandes docker

docker container ls -a

Affiche tous les containers (même ceux qui sont cachés)

docker rm <container ID>

Supprime un container

docker run -it <pseudo>/<nom>:<tag>

Exécute le container en mode interactif (util pour un os ! pas de CMD)

Exemple PostgreSQL

```
# Image officielle Postgres
FROM postgres:16-alpine

# Configuration
ENV POSTGRES_USER=admin_ia
ENV POSTGRES_PASSWORD=admin_pass
ENV POSTGRES_DB=db_ia

# Copie de init.sql pour auto-initialiser
COPY ./init.sql /docker-entrypoint-initdb.d/

# Port standard Postgres
EXPOSE 5432
```

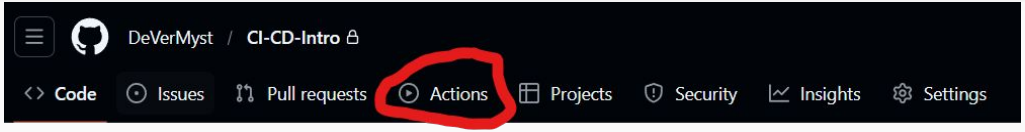
On ne gère pas les variables d'environnement ici, on le fera lorsque l'on fera de l'orchestration de conteneur

Déploiement Continu - CD

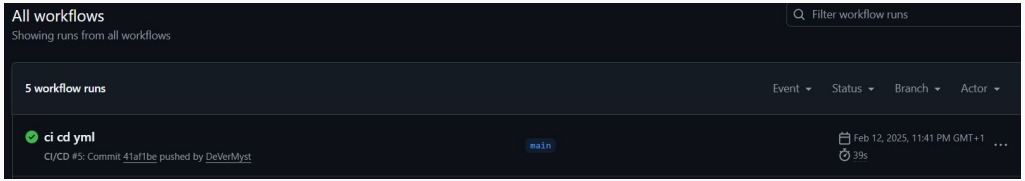
GitHub offre le service de GitHub action qui déclenche automatiquement des actions pré configuré au moment des `push` ou `pull request`.

note : avec une extension, on peut aussi avoir des actions au moment des `commits`

`.pre-commit-config.yaml`




```
.
├── .github/          # Dossier caché de configuration GitHub
│   ├── workflows/    # Dossier contenant les fichiers de CD
│   │   └── cd.yml    # Recette d'automatisation
```



CD d’un Docker simple - cd.yml

```yaml
name: Continuous Deployment (Docker Push)

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
```

CD d’un Docker simple

```yaml
steps:
  - name: Checkout code
    uses: actions/checkout@v4

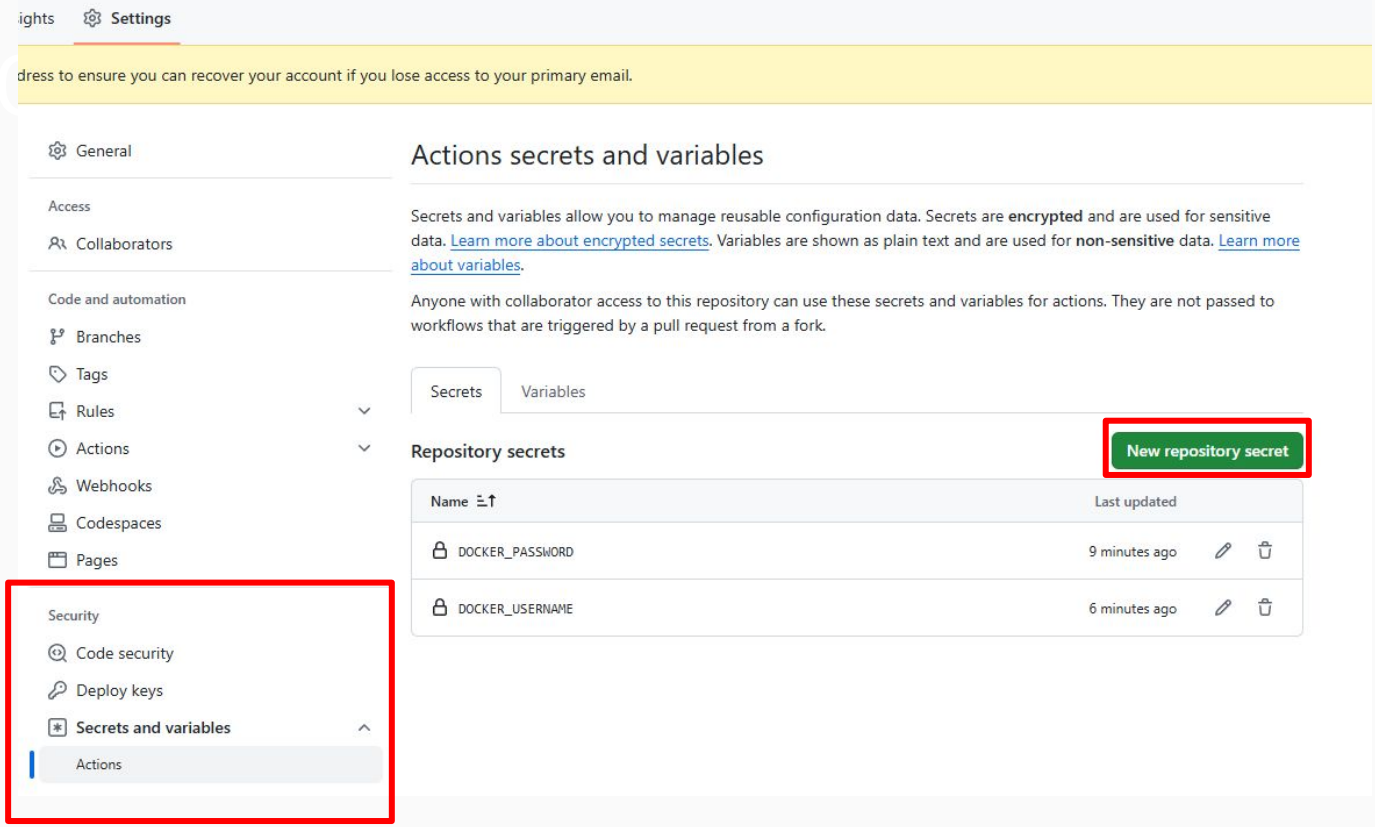
  - name: Login to Docker Hub
    uses: docker/login-action@v3
    with:
      # On utilise des secrets pour ne pas afficher tes identifiants
      username: ${{ secrets.DOCKERHUB_USERNAME }}
      password: ${{ secrets.DOCKERHUB_TOKEN }}

  - name: Set up Docker Buildx
```

CD d’un Docker simple

```
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build and Push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          # Remplace 'DOCKERHUB_USERNAME' par ton pseudo Docker Hub
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/test:latest, ${{
secrets.DOCKERHUB_USERNAME }}/test:${{ github.sha }}
```


ights
Settings
dress to ensure you can recover your account if you lose access to your primary email.
General
Access
Collaborators
Code and automation
Branches
Tags
Rules
Actions
Webhooks
Codespaces
Pages
Security
Code security
Deploy keys
Secrets and variables
Actions
Actions secrets and variables
Secrets and variables allow you to manage reusable configuration data. Secrets are encrypted and are used for sensitive data. Learn more about encrypted secrets. Variables are shown as plain text and are used for non-sensitive data. Learn more about variables.
Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.
Secrets
Variables
Repository secrets
New repository secret
Name
Last updated
DOCKER_PASSWORD
9 minutes ago
DOCKER_USERNAME
6 minutes ago